IA-2 · INTRODUCTION TO OPTIMIZATION TECHNIQUES

Smarter restarts for a hard feed-blending problem

Recipe-Guided Iterated Local Search around SLP for the Fixed-Demand Pooling Problem

Group OPT-09 · Razzan Tambe (15) · Saurabh Tripathi (04) · Vivek Sharma (12)

Dept. of IT, K. J. Somaiya College of Engineering

The problem: blending feed at the lowest cost

Raw materials

The ingredients. Each has a price and a nutrient content.

Mixing bins

A few shared mixes. Many products use the same bin.

Products

Each has a fixed tonnage and a minimum and maximum for every nutrient.

Some ingredients can also go straight into a product, at extra cost.

Goal: the lowest total cost of raw materials, with every product inside its nutrient limits.

Why it is hard

THE CATCH

Product quality = bin share × bin quality

The bin quality depends on the bin recipe, and we choose the bin recipe too.

Unknown × unknown gives many “local best” answers.

THINK OF A HIKER IN FOG

Many valleys. Walking downhill stops in the nearest one, not always the deepest.

96

local best answers in a tiny example with only 2 nutrients

Mills re-solve this many times a day, so they need good recipes in seconds.

Today’s method and its weakness

HOW IT WORKS TODAY

1. Pick a random starting recipe.
2. Run SLP: small, safe steps downhill.
3. Repeat many times and keep the best.

Paper: 863× faster than FilterSQP and 20× faster than the global solver Baron.

RANDOM STARTS THAT END “GOOD”

6.4%

of runs on the 6 hard instances (paper: 14.8%). Good = feasible and within 0.2% of the best known cost.

Runs are independent: nothing is learned from earlier runs.

25–40% of runs on af-4, af-6-f and af-7 end infeasible: they break a limit.

Our idea: three easy calculations

Ignore bin sharing

Diet LP

Each product gets its ideal cheapest recipe. This also gives a lower bound on the best cost.

Bin recipes fixed

Product LP

With the bin recipes fixed, the best product recipes are easy to find, and exact.

Bin shares fixed

Bin LP

With the bin shares fixed, the best bin recipes are easy to find. Alternating both LPs never raises the cost.

On af-1 the Diet LP bound (295.93) equals Baron's 2-hour bound. LP = linear program: solved exactly and fast.

The method in six steps

1 — New

Ideal recipes

Diet LP: each product's ideal recipe.

2 — Modified

Smart start

Group similar recipes into starting bins.

3 — New

Repair

Product and bin LPs make the start feasible.

4 — Retained

Filter-SLP

The IA-1 solver, unchanged.

5 — New

Polish

LPs clean up the end point. Never costlier.

6 — New

Learn from the best

Nudge a bin, test 4 options, keep the best.

The first 3 runs, and any run after 6 without improvement, restart with a fresh smart start. One pass = one run; 20 runs per instance.

The smart move: learn from regret

1. Pick a product with high regret. It pays the most for sharing.
2. Pick a bin and blend its recipe toward that product's ideal recipe.
3. Build 4 such candidates, repair each, and test each with a short 10-iteration SLP.
4. Continue the full SLP from the best candidate.

WHAT IS REGRET?

How much extra a product pays because it must share bins, compared with its own ideal recipe.

WHY TEST FIRST?

3 of 56

On af-3, only 3 of 56 possible moves escaped a stuck solution. A short SLP test ranked one of them first.

How we tested

11

feed-mill instances (af-series)

3 × 20

seeds × runs per instance

1,320

runs in total (+440 ablation)

Feasible	The run ends without breaking any limit.
Good	Feasible, and within 0.2% of the best known cost.
ETiGood	Expected time to get one good solution: time per run ÷ share of good runs.

Same code, machine and seeds for both methods. Significance: Fisher and Mann–Whitney tests.

Results: more good answers, faster

96%

of runs end feasible

Up from 88% with random starts (mean of 11 instances).

33.9%

good runs on the 6 hard instances

Up from 6.4%. The paper reports 14.8%.

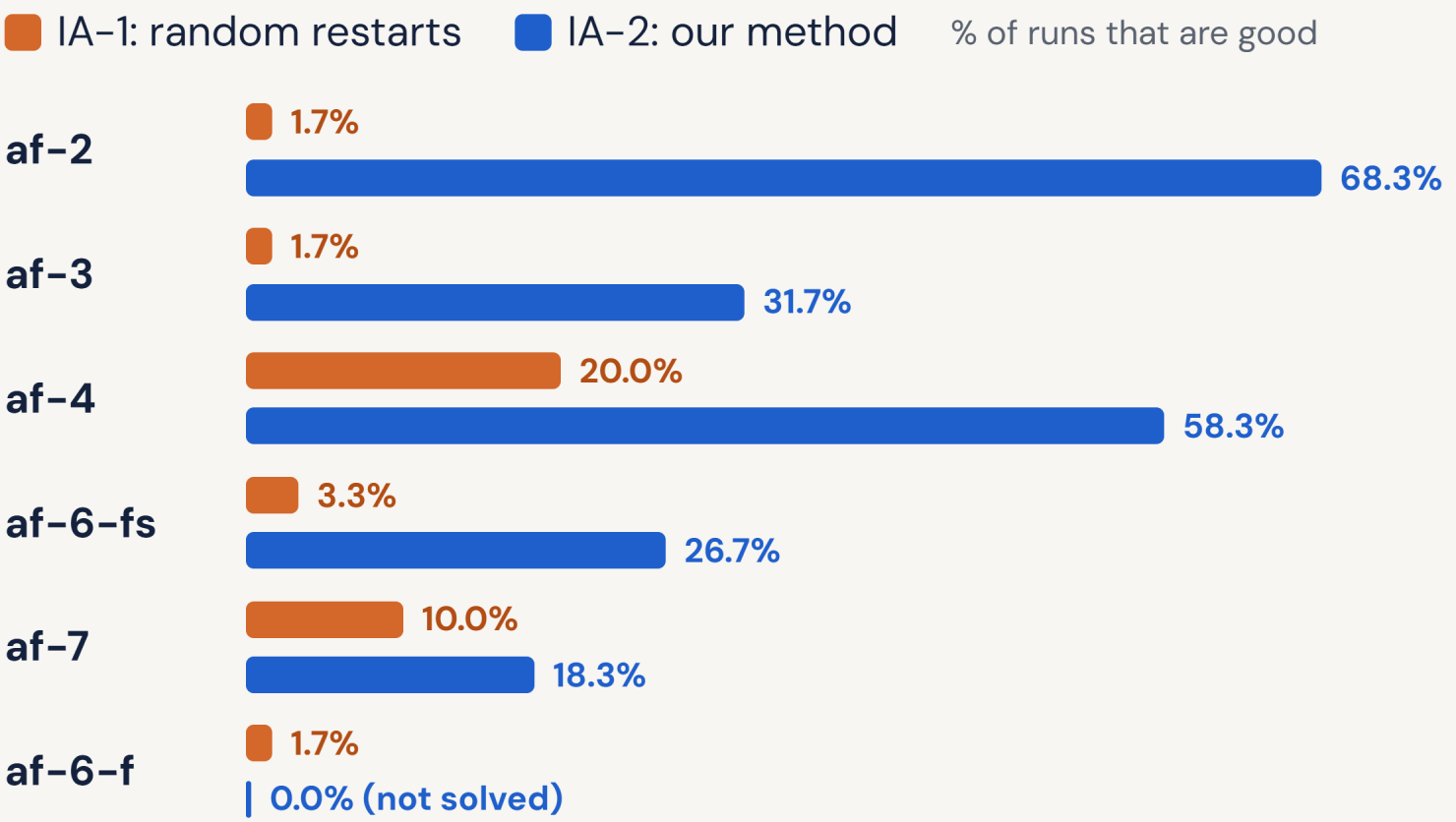
5.0×

shorter time to a good solution

Over 5 of the 6 hard instances. Over all instances with good runs: 2.0×.

Mean over all 11 instances: good runs rise from 43.6% to 62.0%.

Good answers on the hard instances



Paper (SLP-CPLEX): af-2 19.4% · af-3 20.0% · af-4 19.8% · af-6-fs 5.2% · af-7 20.4% · af-6-f 4.0%

Every part of the method helps

NOT JUST LUCK

5 of 11

instances with significantly more good runs (Fisher, $p < 0.01$)

9 of 11

instances with significantly lower final cost gaps (Mann–Whitney, $p < 0.01$)

GOOD RUNS (%), SEED 12345, 20 RUNS EACH

Instance	IA-1 baseline	Polish only	Smart start + polish	Full method
af-2	0%	0%	20%	65%
af-3	0%	0%	5%	15%
af-4	25%	25%	25%	60%

Polish mainly fixes feasibility. Smart starts add variety. The learning search adds the big gains.

Honest limits

af-6-f is not solved

Neither method reaches the good zone: IA-2 0%, IA-1 1.7%, paper 4.0%.

Easy instances get slower

The extra LPs make the time to a good solution up to about 2× worse.

af-7 stays just below the paper

18.3% good runs, against 20.4% in the paper.

More reliable, not always best

On af-4 one IA-1 run found the exact best known cost; our best run is 0.19% above it.

Also: 3 × 20 runs against 500 in the paper, a different LP solver (HiGHS instead of CPLEX), and the diet bound is tight only on af-1.

CONCLUSION

Structure-aware restarts make SLP reliable on hard feed-mill problems.

96%
of runs feasible (was 88%)

33.9%
good runs on hard instances (was 6.4%)

5.0×
shorter time to a good solution

Next: more seeds or 500 runs, adaptive choice between smart starts and learning moves, parallel tests, stronger lower bounds, bin capacities.

Thank you. Questions?